

Project Proposal-OOP

Semester: 2 **Section:** BS-SE 2B **Team #:** 6

Project Title: Hospital Management System

Project Domain (academia, finance, healthcare, etc.): Healthcare

Team Members: (Highlight Team Lead Name + Roll)

<u>Zainab Farooq</u> (Team Lead)	<u>25L-3048</u>
Aliha Saeed	25L-3015
Zuha Shahzad	25L-3016
M.Mueez Ashfaq	25L-3054
Shauzab Mukhtar	25L-3084

Abstract (less than 250 words):

The healthcare industry demands precise coordination across multiple departments, and any inefficiency in managing hospital operations can directly impact patient outcomes. Conventional hospital administration methods, which rely heavily on manual documentation and fragmented record-keeping systems, are increasingly inadequate in meeting the demands of modern medical facilities. To address this gap, the Centralized Hospital Management System has been developed as a comprehensive C++ Windows Forms (WinForms) desktop application aimed at digitizing and automating the full spectrum of hospital operations.

The system is designed in accordance with the principles of Object-Oriented Programming (OOP), incorporating encapsulation, inheritance, and polymorphism to

establish a modular and maintainable software structure. A generalized Person base class forms the foundation of the class hierarchy, from which domain-specific entities such as Patient, Doctor, and Staff are derived. This hierarchical design facilitates structured data management across all user categories within the system. Furthermore, Role-Based Access Control (RBAC) has been integrated to regulate system access, ensuring that each user is granted permissions strictly aligned with their designated role and responsibilities.

For data persistence, the system employs C++ file handling mechanisms through the fstream library, enabling reliable storage and retrieval of records without dependency on an external database management system. The application encompasses ten core operational modules, including patient registration, appointment scheduling, physician and staff administration, electronic medical records, billing and invoicing, pharmaceutical inventory management, room and bed allocation, and emergency ambulance dispatch coordination.

Collectively, these modules form an integrated digital ecosystem that enhances interdepartmental communication, minimizes administrative error, and significantly improves the overall efficiency of hospital management. This project demonstrates the practical application of software engineering principles in addressing real-world healthcare challenges.

List of 8 to 10 Main Features:

1. **Patient Management** — Register, store, and manage patient personal and medical profiles.
2. **Appointment Management** — Schedule, modify, and track patient appointments with doctors.
3. **Doctor Management** — Maintain doctor profiles, specializations, and availability.
4. **Medical Records** — Store and retrieve patient diagnoses, prescriptions, and treatment history.
5. **Staff Management** — Handle non-doctor staff data including roles, shifts, and assignments.
6. **Billing** — Generate and manage invoices for medical services and treatments.
7. **Pharmacy** — Track medicine inventory and process prescription-based drug dispensing.
8. **Room Management** — Allocate and monitor hospital rooms and bed availability.

9. **Ambulance and Emergency Management** — Vehicle dispatch, emergency severity and patient transport coordination.
10. **User Management** — Control system access, user roles, and authentication for all staff.

OOP Concepts Used in the Project:

Encapsulation: Patient, Doctor, and Staff data is stored in private attributes accessible only through public getter and setter methods, preventing unauthorized direct access to sensitive medical and personal information.

Inheritance: A base Person class is defined with common attributes such as name, ID, and contact information, from which specialized classes including Patient, Doctor, and Staff are derived, promoting code reusability and reducing redundancy.

Abstraction: Each hospital module such as Billing, Pharmacy, Appointment, and Room Management is implemented as a separate class, hiding internal complexity behind simple, well-defined public interfaces.

Polymorphism: Virtual display and processing functions are overridden across derived classes to generate role-specific dashboards and outputs for Doctors, Staff, and Patients respectively.

File Handling: Persistent storage is achieved using C++ fstream, enabling serialization of all records including patient profiles, appointments, billing invoices, and medical histories across sessions without an external database.